

Reclamation as a Per-Site Contract

A Four-Rung Memory-Management Ladder Whose Composition Is a Compile-Time Number

The Loment Project · FujioOS

Design paper — October 2026

Abstract. Mainstream garbage collectors are whole-heap algorithms: one policy governs every allocation, and the policy’s behaviour — pause distribution, throughput, footprint — is a *runtime* phenomenon, observable only by running the program. This paper presents an alternative design in which reclamation is a *per-site contract*. Every alloc site is proven, at compile time, onto one rung of a four-rung ladder: **L0** static promotion (the buffer never reaches the heap), **L1** definite lifetime (a `free` is inserted after the last use), **L2** block epoch (batch, non-scanning reclamation at the end of a loop body), and **L3** a runtime collector that receives only the residual. The design’s narrow novelty is not a faster collector but a change in *where* a collector is observable: the **composition** of the ladder — how many sites land on each rung — is emitted into the compilation artifact, making “how much of this program’s memory is not the collector’s business” a number that can be checked without reading a line of source. We give the rule of each rung, the cost written beside each rung, an item-by-item account of what the automatic tier borrows from mainstream collectors and what it refuses and why, and a set of falsifiability criteria in which every rung is refuted by a *pair* of programs differing only in a single configuration line. Finally we close four seductive roads — reversible reclamation, unobservability as reachability, rate-distortion, and thermodynamics-as-design — with reasons rather than deferrals.

1 Introduction

A garbage collector is usually discussed, designed, and benchmarked as a single object: *the* collector. It owns the whole heap, it runs as a policy, and its qualities — a stop-the-world pause histogram, a throughput ratio, a memory overhead, a generational hypothesis — are properties of an execution. You learn what your collector does by running your program and watching it. The design space is correspondingly one-dimensional: given a workload, pick the policy that produces the best curve.

This paper describes a different shape, one in which the collector is not the unit of design at all. The unit is the **allocation site**. Every site where a program asks for heap memory is classified, at compile time, by a proof about that site’s use of the memory it receives. The proofs are arranged as a ladder of four rungs, ordered from strongest to weakest. A site is placed on the strongest rung whose proof it satisfies. The rungs are:

1. **L0 — static promotion.** The allocation is provably bounded and provably contained; the buffer is placed on the stack and never enters the heap at all. There is no reclamation problem because there is nothing to reclaim.
2. **L1 — definite lifetime.** The allocation is provably reachable through exactly one name that never escapes; its last use is countable, and a `free` is inserted immediately after it.
3. **L2 — block epoch.** The allocation lives inside a loop body and provably does not leave it; the body’s allocations are reclaimed in one batch at the end of the body, without scanning.
4. **L3 — the residual collector.** Everything the compiler could not prove is handed to a runtime collector. That is precisely the set of allocations nobody was willing to claim.

The design’s contribution is not a mechanism — every rung has prior art (Section 2). The contribution is a change in **what is observable and where**. In this design the **composition** of the ladder is itself a compilation output. A build of a unit carries a small record:

```
gc_ladder = { l0: 5, l1: 2, l2: 1, l3: 8, total_sites: 16 }
```

and the record obeys a self-consistency rule: the four counts must sum to the unit’s total allocation-site count. A checker can verify that sum without reading the source, exactly as a checker can verify a checksum without understanding the file. The statement

of this program’s allocation sites, five never reach the heap, two are freed at a definite point, one is reclaimed in a batch, and eight are the collector’s problem.

is therefore a **compile-time fact about the artifact**, not a runtime observation about an execution.

The rest of the paper is organized as follows. Section 2 establishes that the ladder is inherited, and states the narrowness of the novelty in one sentence. Section 3 argues the compositional claim, and connects it to the two existing ideas in the same project from which it is drawn. Section 4 specifies the four rungs: what each proves, who reclaims, and the cost written beside it. Section 6 maps three configuration modes onto the ladder and shows that two of them are separated by a **pair of programs**. Section 7 is an item-by-item account of what the automatic mode borrows from mainstream collectors and what it refuses. Section 9 collects the design’s deliberately extreme cells, sorted by falsifiability. Section 10 turns each cell into a refutation test. Section 11 details four roads the design has closed, with reasons. Section 12 is the honest boundary of the design. Section 13 concludes.

1.1 A note on the kind of paper this is

This is a **design** paper. It specifies a design and the conditions under which each part of it would be refuted. It does not report a performance evaluation, and where the design’s width over real code is not yet known, it says so (Section 12) rather than estimating. The virtue it claims is architectural: the design is stated so that each cell is a claim with a price, and each price is a test. Appendix B is evidence about the process rather than the design: how this artifact was produced, and why the toolchain that produced it is visible in the artifact’s own surface.

2 Background: the ladder is not an invention

The ladder is inherited in every rung; Table 1 records the debt.

Rung	Prior work
L0 — stack promotion	Escape analysis and stack allocation, as performed by mainstream JITs and by the compilers of Go and Swift: an allocation that does not escape its frame is placed on the stack.
L1 — inserting free at a definite point	Compile-time release insertion: region inference and its descendants [1], linear and uniqueness types, and Koka’s Perceus reference-count discipline with reuse [2].
L2 — epoch / region batch reclamation	Region inference, MLKit, and arenas. Notably, Baker’s 1992 study already writes the stack discipline and the “collection is a reversed sub-computation” idea into its abstract [3].
L3 — precise root tables	Static types make a precise collector possible; the root set of a typed program is known to the compiler. The caveat that makes this rung nontrivial here is discussed in Section 4.5.
The four-rung composition	Layered and hybrid memory management — a static tier over a dynamic one — is a mature direction, not a new one.

Table 1: Every rung of the ladder has prior art. The design adds no mechanism; it adds an accounting.

The design has no new mechanism. If the reader remembers one sentence from this section, it should be that one. The narrow novelty is stated in Section 3, and it is deliberately small.

Two antecedents deserve emphasis because they bound what is and is not new. First, Baker’s **NREVERSAL of fortune** [3] is the origin of the “reclamation as reverse computation” framing, and it puts the stack in its abstract. Anything in this paper that resembles “reclaim by undoing” is a rediscovery, not a discovery — a fact that matters in Section 11, where that road is closed. Second, the **unified theory of garbage collection** [4] makes precise the duality between tracing and reference counting; the ladder here is not a new point in that duality but a way of **eliminating** sites from it. Region inference [1] is the closest single antecedent: it, too, assigns allocations to statically determined lifetimes. The difference is not in the assignment rule but in what is done with the result, which is the subject of the next section.

3 The narrow novelty: composition as a compile-time number

Mainstream collection *strategy* is a runtime phenomenon. You observe it by running: a pause distribution, a throughput figure, a footprint. This design moves the observability of the strategy down a layer: the strategy’s **composition** becomes a compilation output. Which sites went static, which were given a definite free, which were batched into an epoch, which were left to the collector — all four are counts, and all four are emitted into the artifact.

The claim, then, is not “a better collector”. It is:

the composition of a program’s memory management is a number that can be read off the compilation artifact, without reading the source and without running the program.

This is a claim about **observability**, and it is why the design can be checked the way a manifest is checked. If a build reports $l_0 + l_1 + l_2 + l_3 == \text{total_sites}$, a validator that has never seen the program can still reject a record whose parts do not sum to its whole. That is a weak check — it catches a lie about the parts, not a mistake in the proof — but it is the same kind of check that a checksum performs, and it is enough to make the composition a first-class, machine-checkable property of an artifact.

Written as an identity over a unit’s allocation sites, the record is a decomposition of a whole:

$$l_0 + l_1 + l_2 + l_3 = N, \quad \text{where } N \text{ is the unit's number of allocation sites.} \quad (1)$$

Two derived quantities let the design’s claims be stated as arithmetic. The **compile-time coverage** is the share of sites that never reach the collector; the **residual** is its complement:

$$\kappa = \frac{l_0 + l_1 + l_2}{N}, \quad \rho = \frac{l_3}{N} = 1 - \kappa. \quad (2)$$

The strongest cell of Section 9.5 is then the special case $\rho = 0$:

$$l_3 = 0, \quad \text{iff} \quad \text{the artifact contains no collector.} \quad (3)$$

3.1 Two ideas this design is an instance of

The compositional claim is not isolated; it is an instance of two ideas this project already uses.

- **The artifact carries derived facts.** A compilation unit in this project emits a structured record (its **interface object**) describing its exports and its obligations. The ladder counts are one more field in that record, not a separate mechanism. The design therefore inherits the record’s guarantees: the counts travel with the code that produced them.
- **Self-consistency rules over derived facts.** A derived fact is trustworthy in proportion to how many independent constraints it must satisfy. The summation rule above is deliberately of the same shape as an existing rule the project applies to allocation-boundary totals: a validator that cannot read the source can nonetheless detect that a set of numbers is internally inconsistent. The composition is the second field to be checkable this way, and the fact that it is the second is the point — self-consistency checks over derived numbers are a pattern here, not a one-off.

3.2 What the novelty is not

It is not that L0/L1/L2 exist: Section 2 established that they do. It is not that a precise collector exists: they do. It is not that a hybrid of static and dynamic memory management exists: it does. It is not even that the counts are computed: a compiler that performs these optimizations necessarily knows them. The novelty is that the counts are **emitted, declared, and constrained** — that the composition is a first-class artifact rather than an internal statistic. Whether that is enough novelty to matter is exactly the question a referee should ask, and Section 10 is written so the referee can.

3.3 The nearest prior art, and where the difference lies

The claim of Section 3 is this paper’s only claim, so its nearest neighbours should be named rather than left for a reader to supply. Three are close enough that a referee will think of them.

- **Compilers already report what they optimised.** GCC’s `-fopt-info`, Clang’s `-Rpass`, and the JVM’s escape-analysis tracing all emit, on request, the fact that an allocation was eliminated or promoted. What differs is **status and obligation**: those are diagnostics, absent from the build unless asked for, unconstrained, and not part of what the build produces. The claim here is that the partition is a **field with a constraint** — present whether or not anyone asks, and checkable by a party who never sees the source.
- **Collection metadata already travels in artifacts.** A statepoint-based collector can have its pointer map emitted into the intermediate representation and thence into the object file, so “GC metadata inside the artifact” is not new. But such a map records **where the pointers are** — what the collector must scan — not **how the allocations were partitioned among strategies**. The field of Section 3 summarises a decision procedure rather than describing memory.
- **Reproducible builds already constrain derived numbers.** A manifest that must re-derive to the same digest is the same idea applied to bytes: a derived quantity a third party can check without the source. Equation 1 is that idea applied to a four-way partition instead of a hash.

Naming these is not a concession; it locates the contribution. What is claimed is narrower than any of them taken alone and different from all of them in kind: a **required, self-consistent summary of how a unit’s allocations were distributed across reclamation strategies** — and, on the evidence of Section 8, a summary whose value is currently very far from what the design hoped.

4 The four-rung ladder

Every alloc site is proven onto a rung at compile time. Proofs are attempted from strongest to weakest, and the first proof that succeeds wins. The ladder’s shape is: *the more is proven, the less is paid*, with each rung’s price written beside it rather than buried (Figure 1). Throughout, the **arena** is the fixed heap region the allocator serves and its **frontier** is the bump pointer within that region; to **reclaim** a block is to return it to the allocator.

L0 · Static promotion. The buffer is a literal-sized, contained allocation. It is placed on the stack and never enters the heap. Reclaimed by: **the stack**. Cost: a larger stack frame.

L1 · Definite lifetime. The buffer is reachable through one name that never escapes, so its last use is countable. Reclaimed by: **an inserted free**, placed after the last use. Cost: within the range the token stream can prove; a path the placement cannot reach leaks once — a leak, not an error.

L2 · Block epoch. A loop body’s allocations all stay in the body. Reclaimed by: **a batch return of the allocator’s frontier** at the end of the body — no list, no scan. Cost: a whole-cell granularity, plus four stated boundaries (Section 4.4).

L3 · Residual. Nothing above could be proven. Reclaimed by: **the runtime collector**. Cost: the collector’s full cost — and nothing else’s, because everything else has been taken off its hands.

Figure 1: The four-rung ladder. Proofs are tried strongest-first; the first that succeeds places the site. The colour band marks the boundary between compile-time reclamation (blue) and runtime reclamation (red).

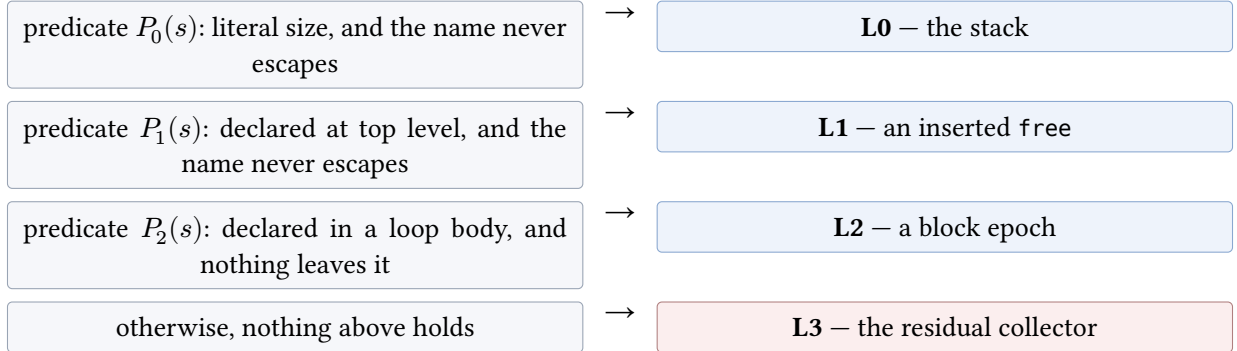


Figure 2: The dispatch rule. A site is placed on the first rung whose predicate holds, tested top to bottom; L3 is the fallback. The predicates are not nested, which is why the design states explicitly that L1 and L2 can never claim the same block (Section 4.2).

The ordering is not arbitrary: it is a **cost** ordering. A site on L0 costs a stack slot and nothing else; a site on L1 costs one inserted call; a site on L2 costs one frontier store per body execution; a site that falls to L3 costs a share of the collector.

With per-rung unit costs $c_0, \dots, c_3$, the ladder’s premise is an ordering of those costs, and what the design actually minimizes is the weighted total:

$$c_0 < c_1 < c_2 \leq c_3, \quad C = \sum_{i=0}^3 c_i l_i. \quad (4)$$

Reducing the l_3 term in Equation 4 is the entire purpose of the design.

4.1 L0 — static promotion, with its rule on the token stream

L0 accepts exactly one syntactic shape:

```
let NAME: ptr = alloc(<integer literal>);
```

where the size is a **literal** — a named constant does not qualify, a hexadecimal literal does — and where NAME, within the enclosing function body, appears in exactly two kinds of position: once immediately after a `let` (the declaration itself), and elsewhere always as **the entire first argument** of a builtin that only dereferences its first argument. The promoted buffer becomes a single entry-block stack allocation that lives for the whole call.

4.1.1 Why the rule is written on the token stream, not the syntax tree

This is the design decision that makes L0 (and L1, and L2) mirrorable, and it is worth stating plainly: the rule is not “walk the AST and check a property”. It is “scan the token span of the function and check a property”. The reason is that the token stream is the only representation that **both** implementations of the language hold in full, and the design requires the two implementations to be byte-identical.

An AST walk has two holes that a second implementation cannot patch:

- Some uses of a name are not in expression position: the binder of `for x in ...`, the variant and binder of a match pattern, the name of a called function, the field names of a struct literal. In an AST these are **strings** inside nodes, and a recursive traversal that walks expressions never sees them — so an AST-based rule would promote **more** than it should. A token scan sees an unclassifiable name, and refuses.
- Conversely, some AST nodes are never recursed into by a given implementation (for example, a dictionary keyed by field name). An AST rule would then see **fewer** uses than exist, and silently **relax** the rule. A silently relaxed rule is the worst failure mode a proof like this can have.

Scanning tokens removes both failure directions. It also forces the rule to be stated in a form that anyone can reimplement with a lexer, which is a virtue in itself.

4.1.2 The dereference-only allowlist

The phrase “only dereferences its first argument” is a property of a fixed, small set of builtins. In the current language exactly three qualify: `load8`, `store8`, and `atomic_add`. Each is lowered inline and does nothing but dereference — it does not store the pointer anywhere, and it does not produce a derived pointer value.

Two exclusions are as important as the inclusions:

- `free` is excluded. It **returns the block to the allocator**; returning a stack buffer to the allocator would be a disaster. A name that is passed to `free` is therefore not dereference-only.
- `ptr_add` and `ptr_sub` are excluded. They **produce a derived pointer value**, and where that value flows is invisible to a local rule. A name passed to pointer arithmetic is not dereference-only either.

The allowlist is the rung’s narrowest point, and it is honest about why. Sixteen-bit and wider loads and stores — `load16`, `load32`, `load64` and their stores — are **not builtins**; they are ordinary public library functions written in terms of `load8` and `store8`. Treating them as safe would be unsound: passing `p` to such a function is a **cross-function call**, `p` enters another frame, and “that function does not stash `p` away” is not something the compiler knows — a user can write a function of the same name and refute the assumption. The correct widening is a cross-function **dereference-only parameter analysis** (if every argument of a function is only ever dereferenced, its first parameter qualifies too), and Section 12 records it as the single widest extension the design is missing.

The honest measure of L0’s width comes from the falsification criteria’s own toy program (Section 10): of sixteen `alloc` sites, five are promoted. That is a toy-scale fraction, not a fraction of real code, and this paper does not pretend otherwise.

4.2 L1 — a free at the point of definite death

L1 accepts the shape

```
let NAME: ptr = alloc(...);
```

with **any** size — literal sizes are L0’s business — provided NAME is declared at the **top level of the function body** (not inside any block), is not a parameter, is assigned once, and every use other than the declaration is exactly the entire first argument of load8, store8, or atomic_add. That last condition is the same non-escape predicate L0 uses, shared between the two rungs: a name that never escapes has exactly one entry point, so a free of it cannot be observed elsewhere.

The top-level-declaration condition is a **safety** requirement, not tidiness. Local slots are allocated in the entry block, and assignment happens only at the declaration. If a declaration were inside a conditional whose branch is not taken, the slot would hold an uninitialized value, and the inserted free would free a garbage pointer. Requiring top-level declaration removes that case entirely.

The placement of the free is a single point: immediately before the earliest return that follows the last use — or at the end of the function when there is no such return (a void function with no use at all also places it at the end). The point is sound for two reasons. It lies after every use **in token order**, and the last use is taken in token order precisely because that order is the conservative one. And once the point is reached, execution does not continue past it: either a return follows immediately — a terminal statement, executed at most once per call even when it sits inside a loop — or the point is the function’s end. A single point that lies after every use and runs at most once therefore cannot free the buffer while a use still lies ahead. Its cost is that any path which leaves the function without passing this point — an early return that precedes the last use — leaks the block once.

Because the placement point is single and executes at most once, and because one function can hold at most one such block, the failure is bounded rather than unbounded:

$$\text{leaked blocks per call} \leq 1. \quad (5)$$

The rung’s boundaries, each with its reason, are set out in Table 2.

L1 boundary	Why it is acceptable
Only dereference-only names qualify	The same allowlist as L0; code that uses load32/store32 library functions cannot use L1 until the cross-function analysis of Section 12 lands.
The declaration must be at function-body top level	Allocations made inside a loop body are not accepted — that is L2’s territory. A stated cost, not a gap.
A single placement point	A path that leaves the function without passing the point — for instance an early return that precedes the last use — leaks once. A leak, not an error.
Cannot overlap L2	L2 requires the allocation to be inside a loop body and L1 requires it at the top level, so the two rungs can never claim the same block. The absence of a double-free path is free.

Table 2: L1’s boundaries, stated as costs.

4.3 Why L1 does not rest on the ownership checker

An earlier draft of this design placed L1 on top of the language’s move checker: a non-Copy value cannot be used after it has been moved out, so a moved-out pointer would be dead and freeable. That draft is wrong, and the correction is instructive. The move checker classifies struct and array types as non-Copy; a **pointer** is Copy. Passing a pointer by value therefore does not count as a move at all, and the move checker says nothing whatsoever about an alloc-ed pointer. L1 is consequently built the same way as L0 and L2 — on the token stream — and not on the ownership boundary at all. The ownership boundary governs what the design declines to prove; it does not license L1.

4.4 L2 — a block epoch the compiler infers

The original sketch of L2 required an explicit epoch construct — which would have meant a new keyword and a change to the language’s frozen surface. The design that survived is more radical in the sense that matters: the epoch boundary is **inferred**, and the language surface does not change at all.

A function body and a loop body are already blocks. If every allocation made inside a loop body provably does not leave that body, then the body’s end can return the allocator’s frontier past all of them at once. The while/for body can therefore open an epoch iff all four of the following hold:

1. the body contains at least one `alloc`;
2. the body contains no `str_concat` — it too allocates, and a `str`’s escape is invisible here, because a string-pointer builtin can hand out an interior pointer as an integer;
3. the body contains no `free`; and
4. every `alloc` in the body has the shape `let NAME: ptr = alloc(...)` (any size), with `NAME` not a parameter, declared exactly once, never reassigned, never appearing after the body, and inside the body appearing only at the declaration or as the entire first argument of `load8/store8/atomic_add`.

Writing $E(b)$ for the indicator that a body b opens an epoch, the four conditions amount to a single conjunction whose failure is all-or-nothing:

$$E(b) = 1, \text{ iff every allocation declared in } b \text{ is contained in } b. \quad (6)$$

The boundaries this imposes are set out in Table 3, and the figure after them draws the all-or-nothing behaviour.

L2 boundary	Why it is acceptable
Whole-cell cancellation	A single allocation escaping the body cancels the epoch for the whole body, not just for that allocation. Finer granularity would require lifetime information — work for L1 or L3.
Only after the body is examined, not before	A name cannot be used before the body that declares it (scoping), so an occurrence before the body does not count as an escape. This is the conservative half that can be done.
Same-named loops block each other	Any occurrence of the name after the body cancels the epoch, even if it belongs to a different loop’s local. On a token stream the compiler cannot tell which <code>p</code> is meant, so it is conservative. The true fix is scope resolution (Section 12).
Names that L0 already promoted are exempt	A promoted name never touches the arena: it neither owes this return nor should it block neighbours that really are on the heap.

Table 3: L2’s four boundaries, stated as costs.

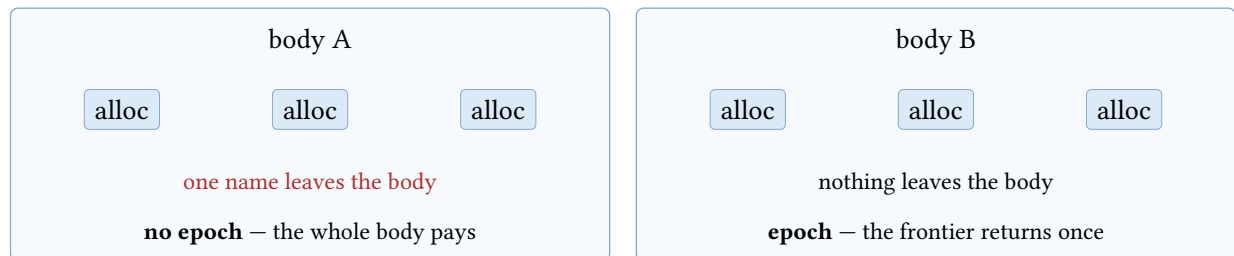


Figure 1: The price of L2, drawn. The epoch is all-or-nothing per body: one escape cancels it for the entire body (Equation 6), and the cancelled allocations fall through to L3.

Why condition 3 (no `free` in the body) is not merely a convenience, but a consequence of a hard toolchain constraint, is worth spelling out, because it shows the design bending to a lower layer. If a `free` were allowed in the body, proving the frontier return would require taking a `min` — the frontier must return to

the **minimum** of the body’s start and wherever frees may have pushed it. A min needs a conditional select. But the project’s in-package mirror linker — the one used to link the objects this design targets — has no select in its lowering. So the design cannot express the conditional if it wants its output to be linkable by that linker. The resolution is condition 3: no free in the body, which makes the argument two steps long and reduces the frontier return to a **single store**. What looked like a small optimization is, in fact, forced into its present shape by the instruction subset of the linker. That the design prefers to **narrow** rather than to **extend the linker** is itself a design choice, and the narrowing is documented as a price rather than hidden.

4.4.1 L2 is stacked on L0, and order matters

L0 runs first and takes the literal-sized, contained allocations off the heap entirely. L2 then returns the frontier past the remainder of the body. The two are additive, and the order is significant: L0 removes sites from the arena’s accounting before L2 batches what remains. The **residual** handed to L3 is therefore “the allocations that are neither of literal constant size nor dead at the end of their own block” — which is exactly what the composition number of Section 3 reports.

4.5 L3 — the residual, and why it is small by construction

L3 is a runtime collector, and the design’s claim about it is directional rather than algorithmic:

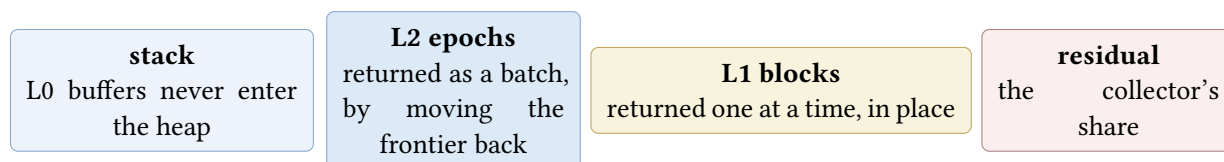
the collector does not **discover** garbage. Everything provably dead has already been released by L0, L1, and L2. What arrives at the collector is exactly the set of allocations that nobody was willing to claim.

The collector’s responsibility therefore shrinks from “the whole heap” to “the residual” — and the residual’s fraction is a compile-time known number, the l3 count in the record of Section 3. This is the sense in which Section 3’s claim is not a performance claim: it is a claim about how much of the memory-management problem remains at runtime, and that quantity is reported by the compiler.

The precise-root question belongs here, and it has a boundary that the design states rather than conceals. A pointer-typed local slot is known to the compiler, so the **root set** can be exact: the compiler emits a slot table listing the addresses of the slots that hold pointers. But a block’s **interior** cannot be scanned exactly, because alloc(N) is untyped — a program may write words into the block with store8, store32, or store64, and the compiler does not know which of those words are pointers. So the interior is scanned **conservatively**: every eight-byte word is treated as a possible pointer. The design’s precise statement is therefore:

“precise collection” holds for the **roots** and not for the **block interior**. To make the interior precise, alloc must carry a type — a separate decision, and one that may require a new lowest-level primitive.

Conservative interior scanning means a block containing an integer that **looks like** a pointer stays alive for one extra cycle. That is an **imprecise** collection, not an **unsafe** one; the design chooses imprecision over unsafe, and says so.



the arena: 64 KiB, and a single frontier

Figure 2: Where each rung’s memory lives. L0 never reaches the arena at all; L2 and L1 return parts of the arena by different means; whatever is left inside the arena is exactly the residual.

5 Formalization and limits

The ladder has so far been described in prose. This section states it as a function, proves the safety of each compile-time rung, and then states what the ladder provably **cannot** do.

5.1 The ladder as a function

Let S be a unit's set of allocation sites, $N = |S|$, and let P_0, P_1, P_2 be the three predicates of Section 4.1, Section 4.2 and Section 4.4 respectively, with P_3 the constant predicate that always holds. The rung assignment $\rho : S \rightarrow \{0, 1, 2, 3\}$ is

$$\rho(s) = i \quad \text{for the least } i \text{ with } P_i(s) = 1. \quad (7)$$

The counts of Equation 1 are then $l_i = |\rho^{-1}(i)|$, so the identity $\sum_{i=0}^3 l_i = N$ holds **by construction** rather than by inspection. This is what the self-consistency check of Section 3 verifies on the artifact: not that the numbers were computed correctly, but that the artifact exhibits a function into a four-element set.

5.2 Soundness of the compile-time rungs

For each rung $i < 3$ we want: if a site is placed there, the rung's reclamation mechanism does not invalidate a live block.

Lemma 1 (L0 is sound). If $P_0(s)$ holds, no value derived from the allocated block ever leaves the frame.

Proof sketch. Every occurrence of the name is, by P_0 , either its declaration or the whole first argument of one of three builtins. Each of those builtins returns a scalar (u32) and stores nothing; none accepts a pointer as a non-first argument; the name is not a parameter and is never reassigned. A pointer value can therefore only be produced by the allocation itself and consumed immediately at a deref site, so the address is never written into any location other than the frame's own slot. A frame-allocated buffer is consequently indistinguishable from a local array, and its lifetime is the call's. ■

Lemma 2 (L1 is sound). If the insertion point p is either immediately before a return or the function's final statement, then no execution of p is followed, in the same call, by a dereference of the block. **Proof.** A return executes at most once per call and transfers control out of the function; when p is the final statement, nothing follows it at all. Hence after p executes there is no instruction of this call left to dereference anything. ■

Corollary (the last-use condition is not a safety condition). Lemma 2 does not use the “after the last use” clause of Section 4.2. That clause is therefore a **liveness** condition — it maximizes the interval reclaimed — and not a safety condition. The rule could free earlier (before an earlier return) without becoming unsound; it would only leak less. We record this because it sharpens what the rung is trading: its stated conservatism is about coverage, not correctness.

Lemma 3 (L2 is sound). Let b be a body satisfying the four conditions of Section 4.4, and let f_0 be the allocator's frontier on entry to b . Then returning the frontier to f_0 at the end of b discards exactly the blocks b allocated, and no live block. **Proof sketch.** Allocation is LIFO, so the blocks declared in b occupy a contiguous suffix above f_0 . Two invariants give the result: the frontier never falls below f_0 while b runs, and no pointer into the discarded suffix survives. The first holds because the only operation that can lower the frontier is a release of the block currently at the frontier, and every such block is either allocated in b — which containment says was never handed out, so no callee holds it — or allocated before b , which cannot lie above f_0 . The second is containment. ■

5.3 Three limits

Theorem A (no exact ladder). Assume the language is computationally universal. Then no computable function decides, for an arbitrary site s , whether $P_1(s)$ holds. Consequently the rung assignment of Equation 7 is necessarily an **approximation**: for every implementation there exist units with sites that are semantically reclaimable at a compile-time rung and are nonetheless placed at L3. **Proof sketch.** Given a Turing machine M , build a unit that simulates M and, on halting, hands a site's pointer to another variable that outlives the site's block. The site escapes if and only if M halts, so deciding escape decides halting. ■

The consequence deserves to be stated plainly, because it reframes the paper's own honesty sections: **the residual is not an implementation deficiency**. No ladder — this one or any successor — can drive κ to 1 for all programs. What a design can choose is **where** the approximation falls, and it should say so, which is what Section 4.1.2 and Section 12 do.

Theorem B (the epoch is strictly LIFO). There is a loop body whose allocations no assignment to L0, L1, L2 reclaims, yet which an L3 collector reclaims. **Proof.** Take a body that allocates a block and assigns the pointer to a variable that outlives the body. Containment fails, so $E(b) = 0$ by Equation 6, and the block is live past the body’s end, so neither a rung-1 free at a definite point nor a frame-allocated buffer is available. The block is reachable but not compile-time reclaimable, which is exactly L3’s remit. ■

Corollary (what “no collector” characterises). Combining Theorem A with Equation 8: since a live set larger than the arena cannot be served without reclamation **during** execution, $\kappa = 1$ is achievable only for units whose peak live set fits the arena. The cell of Section 9.5 — a build with no collector in it — therefore characterises a decidable **class**, not a lucky outcome.

6 Configuration modes and the ladder

The ladder is the mechanism; a small set of configuration modes selects how much of it is active. A program declares its mode with a choose line, or omits it for the default. The three modes and their relationship to the rungs are shown in Table 4:

Mode	L0	L1	L2	L3	Character
gc_manual	—	—	—	—	The default. The program manages memory itself. It must be byte-identical to the behaviour that existed before the ladder was introduced.
gc_auto	—	—	—	mark-sweep	A conventional automatic collector, with the explicit, itemized feature set of Section 7. Zero annotation.
gc_auto_alpha	on	on	on	residual	The whole ladder. The only mode in which the composition of Section 3 is non-trivial.

Table 4: The three modes and the rungs each activates. gc_manual is the default and does not change; gc_auto is the automatic baseline; gc_auto_alpha is the full ladder.

One note on the L3 column: it names the **runtime reclaimer**, if any — a collector under gc_auto, the program itself under gc_manual. The l3 **count** in the record is a different thing: it counts the sites that no compile-time rung claimed, and it is a static fact independent of that column — which is why a gc_manual build reports l3 = total while running no collector at all.

gc_auto_alpha	L0	L1	L2	L3
gc_auto	L3 — all sites residual			
gc_manual	L3 — all sites residual			

Figure 3: The same program, two modes, two compositions. Under gc_auto_alpha the four sites of the Appendix A example spread across the rungs, one per rung; under either other mode every site is residual.

This is what Equation 1 looks like as a picture.

Two properties of this table carry weight.

First, gc_manual moves nothing. The default mode is required to be byte-identical to the program’s behaviour without the ladder — the ladder is an **addition** to the language, never a change to the meaning of existing programs. This is a hard constraint on the design, and it is what allows the two later modes to be compared against an unchanged baseline.

Second, the difference between gc_auto and gc_auto_alpha is **measurable**, and it is not a difference of degree. Same program, two modes, two artifacts that differ in their ladder counts — and, on at least one program, two different **behaviours**: a program that completes under alpha and exhausts the arena under auto. This converts the phrase “a hybrid of static and dynamic reclamation” from an adjective into a **pair of programs**. A design that cannot exhibit such a pair has not designed a hybrid; it has designed a collector with a static tier bolted on that never changes any answer.

6.1 The automatic baseline has a stated ceiling

gc_auto's collector triggers on **bytes allocated** — it collects after roughly 32 KiB has been allocated since the last collection, and resets the counter — while the arena holds 64 KiB. The consequence is a ceiling that the design states rather than conceals:

a program whose **live set** exceeds roughly 32 KB will still exhaust the arena under gc_auto.

With a trigger threshold T and an arena of size A , a live set L survives only if one trigger's worth of allocation still fits beside it:

$$L \leq A - T. \tag{8}$$

For this mode the two constants are $A = 64$ KiB and $T = 32$ KiB, so Equation 8 reads $L \leq 32$ KiB. The ceiling is not a tuning choice; it is that arithmetic.

Raising this ceiling requires the trigger to see how large the live set actually is, which is the generational/adaptive family of techniques. That family is declined for gc_auto, with a reason, in Section 7. The ceiling is not a deferred obligation; it is this mode's stated upper bound, and the honest way to read the mode is as an automatic collector with a declared working-set limit.

7 What the automatic tier borrows, item by item

The automatic mode is specified as the **(union of the merits)** of mainstream automatic collectors [5] — not as one collector's design copied wholesale, but as a chosen set of the strengths of several. Taking that definition seriously means naming, one by one, what is copied, what is refused, and why in each case. Silence is not a reason. Table 5 and Table 6 are the account.

Borrowed method	How it appears here	Cost, written out
Precise roots	The root set is not guessed. The compiler emits a slot table, parameters first and locals in declaration order, holding the addresses of the slots. This is the opposite trade from a conservative collector that scans the stack as raw bytes: a false pointer can never arise at a root.	Every function that holds pointers pushes at entry and pops before every return.
Mark-sweep, non-moving	Clear marks; scan roots; propagate to a fixed point; sweep. Four passes. The oldest cell, and the cheapest.	Fragmentation is not removed (no coalescing); in exchange there is no pointer fix-up, and therefore no pause amplification from moving.
Trigger on bytes allocated	Collect after the counter exceeds a threshold and reset it. This is the mainstream shape — “collect after N bytes since the last collection” — rather than “collect when the heap is full”.	O(1) and predictable; the price is that the trigger cannot see the live set, which is exactly where the ceiling of Section 6.1 comes from.
Collect before allocating	A guard is emitted before each <code>alloc</code> site. This is equivalent to “try to allocate, collect on failure and retry” — the fallback every collector has.	The benefit is that the allocator’s own entry point does not change by a single byte, so the <code>gc_manual</code> and <code>gc_auto_alpha</code> artifacts are byte-for-byte unchanged.
Zero annotation	Not writing a <code>choose</code> line yields the default; writing one yields everything. Compare: Rust’s <code>Rc/Arc</code> and lifetime annotations, C++’s smart pointers, the JVM’s long list of <code>-XX</code> : tuning flags.	The reverse is that there is no per-object policy: fine control means switching modes, not turning a knob inside this one.
Usable immediately after collection	Sweeping returns dead blocks to the address-ordered free list; a block that sits at the top of the frontier comes back by frontier roll-back. Both paths already exist in the allocator; the collector merely hands dead blocks back to it.	No extra cost. This is the precondition for “non-moving” being viable at all.

Table 5: What `gc_auto` borrows from mainstream collectors — each entry with its cost.

Refused	Reason
Generational collection [6]	A concrete existing property of this backend: pointers live only in local and parameter slots — the fields of structs and variants hold scalars only. The one thing a write barrier exists to manage, “an old-generation pointer to a young object”, therefore barely exists, and a card table buys almost nothing. Revisit when fields can hold pointers.
Moving / compaction	Not laziness — impossibility. A block’s interior is scanned conservatively because <code>alloc(N)</code> is untyped; moving a block requires knowing the type of every word in it, and “conservative” is precisely “we do not know”. Conservative scanning and moving are welded together: take both or neither.
Incremental / concurrent / sub-millisecond [7]	Two reasons. This backend is single-threaded with no signals, so there is no concurrency to exploit. And the qualitative promise — a collection with no phase that freezes the whole workflow — is attached, in this design, to <code>gc_auto_alpha</code> (Section 9), not to <code>gc_auto</code> . <code>gc_auto</code> performs an honest stop-the-world and does not pretend otherwise.
Adaptive strategy pools, lifetime prediction	Those are <code>gc_auto_alpha</code> ’s ambitions. Borrowing them here would erase the distinction between the two modes, which Section 6 insists is measurable.
Weak references / finalizers	The language has no corresponding construct — no weak, no Drop.
Large-object regions / TLABs	There are no threads and no page-management layer to host them.
Coalescing of adjacent free blocks	Absent by an earlier decision and not back-filled by the collector. Coalescing is the allocator’s job; the collector’s job is to hand dead blocks back to the allocator.

Table 6: What `gc_auto` refuses, and why — one reason per line, none deferred.

7.1 The price of the automatic tier, stated

The account would be incomplete without the costs of `gc_auto` itself:

- A unit that uses `alloc/free` carries roughly 5.2 KB of collector text in its artifact; units that do not allocate do not carry it, because the collector is embedded in the “allocation is needed” branch.
- The root table has 1024 slots and, when full, **refuses** — the overflow path aborts rather than silently writing past the end. Only deep recursion is likely to reach it. Writing n for the slots already in use and k for those a frame needs, the emitted guard is $n + k \leq 1024$ (Equation 9), and the false branch of that comparison jumps to an abort.

$$n + k \leq 1024 \tag{9}$$

- A live set above roughly 32 KB exhausts the arena (Section 6.1).
- Because the block interior is scanned conservatively, a block containing an innocuous integer that resembles a pointer stays alive one extra cycle — an imprecision, not an unsoundness.

8 The measured composition

Section 3 claimed that the composition is a compile-time number. This section reports what that number is on real code. A design that asserts a partition owes the reader its size, and here the size is unflattering, which is why it is reported rather than left as an estimate.

8.1 Method

Every .lomt unit in the project’s own source tree was compiled by the project’s reference compiler with its configuration line forced to `gc_auto_alpha` and `runtime`, so that the ladder is active. Nothing else was edited; no unit was rewritten to suit the ladder. Units whose surface grammar is not Loment (the C- and Python-surface units of the multi-grammar corpus, 27 units) and units that do not parse (9 units) were excluded, and both counts are given so that the sample is auditable. The measurement is a **retrofit** one: this corpus predates the ladder, so κ measures how much of existing code the unmodified rules accept, not what a ladder-aware codebase could reach. That is the conservative direction, and it is the one that matters.

8.2 The result

Source area	units	sites	L0	L1	L2	L3	κ
Standard library	128	2968	15	0	0	2953	0.51 %
Package manager	35	113	1	0	0	112	0.88 %
Tooling	29	22	2	0	0	20	9.09 %
Examples	40	29	6	1	1	21	27.59 %
Self-hosted compiler	24	3	0	0	0	3	0 %
Surface-grammar corpus	9	3	0	0	0	3	0 %
Core library	10	1	0	0	0	1	0 %
Total	275	3139	24	1	1	3113	0.83 %

Table 7: Ladder composition over the project’s own Loment-source corpus, with the automatic tier forced on for every unit. The standard library holds 94.6 % of all allocation sites.

3,139 allocation sites — 3,113 of them the collector’s

magnified $\times 120$ — the compile-time sites, against the same span of L3

L0 = 24

L3

L1 = 1, L2 = 1 — one site each, in the entire corpus

Figure 3: The same data, drawn: one full-width bar for the corpus, and a magnification of its left edge. The compile-time rungs are not a small fraction of this corpus; they are a sliver of it.

Headline (Table 7, drawn in Figure 3): **26 of 3,139 sites**, so $\kappa = 0.83\%$ and $\rho = 99.17\%$. The standard library holds 2,968 of the sites, 94.6 % of the mass, and there $\kappa = 0.51\%$. Only 16 of the 275 units place a single site above L3 at all. (The “Examples” row is flattered by a tiny denominator, and it contains the ladder’s own witness unit from Appendix A; excluding that unit the row falls from 27.59 % to 20.0 %.)

8.3 Why, and the cause is one the design already names

The number is not a mystery, and this is the part worth keeping. Two mechanisms, both verified on the largest unit:

- **The size is computed.** `let a: ptr = alloc(bigdec_bytes(16));` — L0 requires a literal size (Section 4.1), and a call is not a literal.
- **Access goes through library functions.** The same unit reads and writes its buffers with `mem_load32` and `mem_store32`, wrappers defined in another unit. The deref-only allowlist contains exactly three builtins; a call to anything else puts the pointer into another frame, which the rung may not assume away (Section 4.1.2).

Where code does use the three builtins directly on a literal-sized, non-escaping buffer, the rung fires: one unit in the standard library places 8 of its 57 sites on L0. The mechanism is live, and narrow, which is precisely the trade Section 4.1.2 predicts.

8.4 What this does to the paper’s claims

It separates the claim that survives from the one that does not.

- **Survives.** The composition is a compile-time number, it is emitted, and it is self-consistent (Equation 1). The measurement is itself the demonstration: κ was obtained by reading artifacts, without running a program.
- **Does not survive.** Any reading in which the ladder “usually” removes allocations from the collector. On this corpus the collector keeps 99.17 % of the sites, so as implemented today the four-rung ladder is a nearly degenerate partition.

That is the design’s ambition falsified by the design’s own instrument — the outcome Section 10 was constructed to produce. It also isolates exactly one next step: the cross-function dereference-only parameter analysis of Section 12, which would address the second mechanism above and is the only change the data argues for.

9 Radical cells

The design has several deliberately extreme consequences. They are presented here sorted by **falsifiability**, not by appeal: the ones that could be wrong in a way a test would catch come first.

9.1 Compilation-time rejection instead of runtime degradation

If the worst case of a residual allocation can be bounded — for example, an allocation inside a loop whose trip count is statically known — and that bound exceeds the arena, the **compiler rejects the program by name** rather than emitting a program that will exhaust memory at runtime.

“Not enough memory” becomes a **compile-time diagnostic** rather than a runtime catastrophe.

The honest boundary is immediate: the worst case of the residual is **usually** not bounded, since allocation counts depend on input. The claim holds only for the subset that **is** bounded — but for that subset, “will this program run out of memory?” is a **decidable question**, which in mainstream languages is not something one can even ask.

9.2 The root table is an auditable artifact

If the root set is a set of pointer-typed local slots, and types are static, then the compiler knows the root set and can **emit** it. Therefore

“what the collector considers live” can be reviewed **without running the program**.

Mainstream collectors’ policies are invisible runtime behaviour; this cell turns them into a table in the artifact. The honest boundary is the one stated in Section 4.5: roots are exact, the block interior is not, because `alloc` is untyped. The design states which half is precise rather than claiming the whole.

9.3 The residual is unclaimed, not discovered

A reversal of direction: the collector does not **discover** garbage. Whatever is provably dead has already been handled by L0/L1/L2. What is handed to the collector is exactly the part nobody was willing to claim. The collector’s remit shrinks from “the whole heap” to “the residual”, and the residual’s proportion is a compile-time number (Section 3). This is the cell that most directly distinguishes the design from “a collector with optimizations in front of it”: in those systems the collector still owns the heap and the optimizations reduce what it **touches**; here the collector owns only the residual by construction.

9.4 The epoch is the only non-scanning batch reclamation — and it buys stack discipline

L2 can avoid scanning because it is **strictly last-in-first-out**: the frontier returns, and an entire contiguous stretch disappears. This is not the design’s invention — Baker’s abstract named the stack [3] — and it is not free: it means that **any reclamation which is not last-in-first-out cannot be expressed in L2**. L2 is therefore a mode that trades **expressive power** for **zero cost**; writing it more prettily would only hide the price.

Landed, the price is concrete. The cost is not “a compile error”; it is that the cell **silently fails to obtain an epoch** (the epoch test returns false), and the allocation falls through to L3 — which, before L3 exists for it, means no reclamation at all. And “which cell obtained an epoch and which did not” is a compile-time known fact, pinned cell by cell in the epoch truth-table of Section 10.

9.5 The norm is “there is no component called GC”

If a program’s L3 residual is zero, the artifact contains **no collector** — and an $L3 == 0$ count in the record of Section 3 is the evidence. “This program does not need a GC” is thus a **decidable fact**, not a hope. The design can already reach half of this: a fully-L0 `gc_auto_alpha` artifact contains not even the allocator, because the compiler includes the allocator runtime only if the emitted text still references it — promote everything and the runtime drops out by itself. The other half — reading it off the $L3 == 0$ count — is exactly what the compositional record provides. This also makes contact with the runtime configuration dimension — the switch that turns the runtime on or off: a build with the runtime enabled but $L3 == 0$ is a build in which **the runtime exists and is not used** — a cell the design can express.

9.6 The target: no phase that freezes the whole workflow

The qualitative goal attached to the full ladder is not a shorter pause. It is the stronger statement that there should be **no phase that needs to freeze the whole workflow**. A sub-millisecond stop-the-world pause is still a stop-the-world pause; the design’s ambition for `gc_auto_alpha` is that the question does not arise, because by the time anything reaches the collector, so much of the work has already been done at compile time that no runtime phase has to halt everything.

This is the least falsifiable cell in this section, and the paper says so. It is realized not by a mechanism but by the **size of the residual**: the smaller the $L3$ count of Section 3, the less there is for any runtime phase to freeze. The falsifiable half of it is therefore the composition number itself — a design whose residual is not small has not earned this claim, and the number says so. `gc_auto`, by contrast, performs an honest stop-the-world and makes no such claim (Section 7).

10 Falsifiability

A design cell is worth having only if it can be shown wrong. Every cell in this design is refuted by a **pair of programs** that are textually identical except for a single choose line. The pairs matter: a test that only shows “the collector reclaims” can be passed by a collector that never collects, and a test that only shows “the pointer is freed” can be passed by a collector that frees everything. The two directions must come as a pair (Table 8).

Cell	The pair that refutes it
Self-consistency of the composition	$l_0 + l_1 + l_2 + l_3 == \text{total_sites}$. A validator that cannot read the source can still reject a record whose parts do not sum to its whole. The rule’s shape is shared with an existing allocation-boundary self-consistency rule in the project, which is why it is trustworthy as a pattern rather than a one-off.
L0 really fires	A pair differing only in choose: under <code>gc_manual</code> all sixteen allocation sites are untouched; under <code>alpha</code> only eleven remain, the five promoted ones replaced by a single stack allocation.
L0 is not overstated	Two hooks, one per direction: with promotion disabled but the count kept , the <code>alpha</code> cell climbs back to sixteen and the test is red; with the count forced to zero while promoting , the eleven-site cell no longer matches and the test is red again.
L1 really fires	A pair whose allocation sits in a helper function and whose loop is in the caller, so that neither L0 (the size is not literal) nor L2 (the body has no <code>alloc</code> , only a call) can reach it and only L1 can. Four thousand calls exceed the arena, so <code>alpha</code> must complete and <code>manual</code> must exhaust — otherwise L1 is refuted. The static half pins, cell by cell on a truth-table, the insert/do-not-insert decision over functions that include the four disqualifying shapes (declaration not at top level, escaping, already user-freed, reassigned).
L2 really fires	A pair whose loop body allocates a dynamically-sized block each iteration, so that L0 cannot apply and the accumulation exceeds the arena by more than an order of magnitude. The peak live set is a single block, so “completes” can only be the frontier roll-back’s doing.
L2’s price is real	The truth-table pins as false exactly the bodies that are not last-in-first-out — a pointer leaving the body, a pointer handed to a function, a <code>free</code> in the body, a <code>str_concat</code> — each with the reason written beside it. Those cells must wait for L3.
L3’s root table is correct	A pair that leaks one block per iteration and keeps exactly one block live until the end, with a final read that acts as a sentinel that the live block was not reclaimed early. Under <code>gc_auto</code> the program completes; under <code>gc_manual</code> it exhausts. The static half is complementary: in <code>gc_auto</code> ’s artifact each <code>alloc</code> site is preceded by exactly one collect guard and the root table registers exactly the expected slots, while the other two modes carry no trace of the collector at all.
The default is unchanged	A corpus with no choose line must produce a byte-identical artifact — an existing invariant of the project, guarding the “additions only” rule of Section 6.
The two implementations agree	The L0 pair, the L1 truth-table, the L2 truth-table, and the collector itself must all be byte-identical between the two implementations of the language. The collector case is the hardest to get right by accident: the collector body can be generated, but its attachment points are hand-written, and each root-table slot consumes several temporaries, so a one-off error in numbering shifts the entire block.

Table 8: Each design cell and the program pair that would refute it. The two directions of each pair are complementary, in the same way as the manual mode’s own falsification pair: a test that runs **with** an inserted `free` and a test that exhausts **without** it.

The complementary structure deserves one more sentence. The “L3 root table is correct” cell is complementary **within itself**: a collector that never collects passes the “does not reclaim the live block” half, and a collector that reclaims indiscriminately passes the “completes” half. Only the **pair** — a leaked block that must not survive, and a live block that must survive — refutes a collector that gets either direction wrong. The same pairing, in the manual mode, is what distinguishes “a free that works” from “a free that frees the wrong thing”.

11 Four closed roads

The most useful part of a design is often the part that closes options, provided the closures come with reasons rather than “later”. Four roads look deep and are closed here.

11.1 Reversible reclamation — closed

The idea: keep enough history to run each step backwards, so that returning a block is literally undoing an allocation, and thereby derive a reclamation scheme from a reversible computation. It is closed, and the reason is not difficulty — it is **aliasing**. To make every step reversible one must keep enough history to invert it, and aliases are everywhere (every mutable local, every store through a pointer, is irreversible). What must then be retained is **the entire computation**, so the roll-back depth grows without bound and memory is unbounded. A bounded roll-back depth is checkpoint/rollback, not the inverse of a program.

The prior art is decisive on this point: Baker’s **NREVERSAL of fortune** [3] already wrote “collection is reversed mutation” and the stack into its abstract. The design’s relationship to that road is rediscovery, and this section records it as such rather than presenting it as something new. (The thermodynamics that usually accompanies this road is a separate closure, Section 11.4.)

11.2 Unobservability as a reclamation criterion — closed

The idea: use the language’s capability domains — the static marks by which a unit declares which spaces it may send data to, in particular the `excluded/closed` predicates — to decide that a block is unreachable, hence freeable. It is closed because the arrow points the wrong way. An observability relation answers “which spaces can this unit **send** data to” — an **outward** permission. Reclamation asks “which pointers can still **reach** this block” — an **inward** reachability. These are opposite directions on the same graph, and no amount of transitivity makes the outward relation into the inward one. Nor can a layer of flow analysis rescue it: such an analysis would say “unobservable from outside, hence discardable”, while the program in question is reading the block through `*a/*b`; to call it discardable one must know the liveness of the **interior** pointers, which is reachability itself, which is where the question started. The road does not lead to a gap; it leads back to L3.

11.3 Rate–distortion — closed

The idea: model garbage as a source and reclamation as a distortion-bounded encoding, and use rate–distortion theory to bound what may be discarded. It is closed because the theory’s content lives at positive distortion. At zero distortion the rate is just the entropy of a quotient [8] — a single point, not a curve — and using it gives “irreducible information” a standard name without contributing a theorem. Genuine rate–distortion reasoning requires the language to carry a **declared error budget**, and a language with a declared error budget is describing approximate storage — a different word than garbage collection.

11.4 Thermodynamics as a design basis — closed

The idea: derive a reclamation scheme from Landauer’s bound [9] and the reversible computation that dodges it [10] — the “erasure costs $k T \ln 2$ ” line of reasoning. It is closed because the tax is attached to **reuse**, not to **reclamation** — the generalized form, in which the cost depends on correlations with the environment [11], does not change this. A `free` erases nothing (it writes a few bytes of a block header); a compacting collector **abandons** rather than erases; what actually dissipates entropy is the **next overwrite**, and the program would have to perform that overwrite anyway. This account therefore cannot distinguish “has a GC” from “has no GC” — it distinguishes **reusing** from **not reusing** — and it is three to six orders of magnitude below a single CMOS switch. If a physical objective function is wanted, the quantities to use are **retention power**

(DRAM refreshing every 64 ms, growing linearly with surviving bytes) and **data movement** (a DRAM access costing two orders of magnitude more than an L1 access) — not entropy.

The four closures above are not a list of things that might be done later. They are roads the design has walked far enough to know the direction, and turned back from.

12 Honest boundaries

A design paper’s honesty is measured by what it declines to claim. The following are boundaries of this design, stated as boundaries.

- **The proofs’ width over real code is now measured, and it is small.** Section 8 reports $\kappa = 0.83\%$ over 3,139 allocation sites in 275 units of the project’s own corpus, and 0.51 % in the standard library, which holds 94.6 % of the mass. Earlier drafts of this paper listed that number as missing; it is now supplied, and it is the least flattering result here. The toy figures quoted in Section 4.1 (sixteen sites, five promoted) remain in place only as the criteria’s worked example.
- **The three rungs share one narrow safety allowlist.** `load8`, `store8`, and `atomic_add`. Code using the wider library loads (`load32/store32`, and so on) cannot obtain L0, L1, or L2. Section 8 identifies this, together with computed allocation sizes, as the dominant cause of the measured 0.83 % coverage. The single widest extension is a cross-function **dereference-only parameter analysis**, and that change is larger than the three rungs it would widen.
- **L2’s boundaries are the price of the token stream.** Whole-cell cancellation, “only after the body”, same-named loops blocking each other, and the `str_concat/free` exclusions are all costs of being unable to recognize scope on a token stream. The true fix is scope resolution — maintaining a name table per block — which is a separate change.
- **L2 recognizes only loop bodies.** `if` bodies and function bodies are also blocks that could open epochs; the design does not yet do so. The function-body case overlaps L1 (definite lifetime plus inserted `free`), and either could come first.
- **Block-interior precision is unattainable without a type on `alloc`.** As Section 4.5 states, roots are precise and interiors are not. Making interiors precise requires `alloc` to carry a type, which may require a new lowest-level primitive.
- **The collector has not been measured on a real corpus.** Every number associated with `gc_auto` comes from the criteria’s toy programs, as with the width question above.
- **The automatic baseline has a working-set ceiling.** A live set above roughly 32 KB exhausts the arena under `gc_auto` (Section 6.1). Raising it needs a live-set-aware trigger, which the design declines for this mode with a reason (Section 7) rather than deferring.

None of these is a hidden obligation. Each is a place where the design, given its own constraints, chose a smaller thing and named it.

13 Conclusion

Loment’s memory management should not be “a better algorithm”. It should be a **classification**: every block of memory assigned, by how much the compiler can prove about it, to a rung of a four-rung ladder; the part that is provably dead never reaches the collector at all; and **how much went to each rung** written into the artifact.

What this yields has no counterpart in the mainstream column, because it is not a shorter pause. It is a **readable composition** — a program’s memory management described by four numbers that a validator can check without the source and that a reader can understand without a profiler. And because every cell of the design is refuted by a pair of programs differing in one line, the composition is not only readable but **testable**, cell by cell.

Appendix A — the ladder in one program

The following program, written in Loment, puts one allocation site on each rung so that all four counts of the record in Section 3 are non-zero. It is the criteria’s concrete witness that the composition is a four-part number rather than a three-part one.

```
// One site per rung of the four-rung ladder.
module gc_ladder

choose gc_auto_alpha
choose runtime

fn main() -> i32 {
  let a: ptr = alloc(64);           // L0: literal size, dereference-only
  store8(a, 0, 9);
  let n: u32 = 8;
  let b: ptr = alloc(n);           // L1: variable size, top-level, dereference-only
  store8(b, 0, 3);
  let i: i32 = 0;
  while i < 4 {
    let c: ptr = alloc(n);         // L2: allocated in a loop body, name never leaves
it
    store8(c, 0, 1);
    i = i + 1;
  }
  let d: ptr = alloc(n);           // L3: escapes (handed to another variable) – nobody
claims it
  let e: ptr = d;
  store8(e, 0, 1);
  return (load8(a, 0) as i32) + (load8(b, 0) as i32) + (load8(e, 0) as i32);
}
```

The four counts this program produces under `gc_auto_alpha` are the composition of Section 3: `l0 = 1`, `l1 = 1`, `l2 = 1`, `l3 = 1`, `total_sites = 4`, summing to the whole. Under a mode other than `gc_auto_alpha`, no site is claimed by a compile-time rung and the record degenerates to `{0, 0, 0, total}` — every site in the runtime column, where a collector reclaims it under `gc_auto` and the program itself does under `gc_manual`. That is the design’s way of saying, in the artifact, “this mode does no compile-time reclamation”.

Design facts in this paper follow the project’s GC design document (docs/210), whose four rungs, three modes, closed roads, and boundaries it restates. This paper is a design specification and a statement of falsifiability conditions; it makes no performance claim.

Appendix B — How this design was produced

This appendix is evidence about the **process**, not part of the GC argument. It is included because the process is unusual enough to be a fact about the artifact: this design, and the language it belongs to, were produced by an autonomous agent toolchain, and that fact is visible in the design itself. Everything below is drawn from the project’s own records, cited by document.

B.1 The work was delegated to an agent, explicitly

The project’s decision record (docs/140 §10, dated 2026-09-08) quotes the instruction that set the mode of work: “develop it all together; make Loment open source; develop until it is completely finished (I will not be present, so you have to push it forward yourself)”. The record resolves it into a rule: advance autonomously, with zcode executing to “completely finished” without asking for a ruling at each step, and deciding for itself when blocked, while recording every such decision. Handoff documents in the project accordingly carry the byline “author: the Loment line (zcode)” (docs/155).

The design in this paper was therefore not produced by a human directing an editor with an assistant attached. It was produced under a standing delegation that carries an obligation to **record**: a blocked decision becomes a document.

B.2 The model that served the sessions

Every claim above names an **agent**. This subsection records the **model**, because on the machine that produced this paper the two are not the same, and the model is not an unstated one.

The Claude Code client that ran these sessions is configured to send its requests to DeepSeek’s Anthropic-compatible endpoint. Three settings in the client’s environment establish this: the base URL is `https://api.deepseek.com/anthropic`; the model bound to every slot (the settings named for “Sonnet”, “Opus” and “Haiku” alike) is `deepseek-flash`; and requests are authenticated with a DeepSeek API token. Client protocol and model therefore come from different vendors: an Anthropic-compatible client on one side, a DeepSeek model on the other.

This also explains an apparent inconsistency a reader may notice. The client announces its model by **slot** rather than by served model, so a session under this configuration describes itself as a Claude model of whatever tier the slot names, while the slot itself resolves to `deepseek-flash`. The name in the banner is a slot; the model is the one named above.

Honest boundary. The repository records the **agent** (zcode) but not the model that served it, so this attribution is a fact about the **development environment**, recorded here by the authors, and not something derivable from the artifact. It is scoped to the sessions run under this configuration; it is not a claim that every session in the project’s history was served the same way.

B.3 The language was shaped by the fact that an agent has to write it

The sharpest AI-facing fact in the project is not that an assistant helped. It is that the **language’s surface was chosen to match what a language model can write**.

docs/110 §1 states the objection: model training is aligned almost entirely with mainstream languages such as Python, so inventing a syntax yields “zero training corpus, zero generation capability, purely self-inflicted obstruction”. docs/140 rebuts it head-on and names the stakes bluntly: “a new language = zero corpus = the agent cannot write it”. The first of its three answers is a design decision. The grammar is not new, only the semantics: Loment’s surface syntax is a strict subset of Rust, so a model’s Rust prior transfers unchanged, with C++ against C, TypeScript against JavaScript and Kotlin against Java cited as precedents. The other two answers move the problem elsewhere, by having the compiler emit a complete machine-readable semantic model so that an agent reads the model rather than the binary (docs/140 §4), and by making the first version of the language transpile rather than compile.

The consequence is easiest to see in the things this paper does **not** explain. The language has no epoch keyword (Section 4.4), no new punctuation, and no new statement forms for the four rungs, not because such

forms were judged inelegant, but because a model writing Loment draws on a corpus for Rust, and every invented form is a word it has never seen.

B.4 The language is also made legible to the agents' editors

A second, smaller effort runs the other way: the tooling is fitted to the agents. A VS Code extension supplies the real editing support, in the form of two TextMate grammars plus an LSP, while the built-in viewers of two agent hosts that could not be extended, namely ZCode's Shiki-based viewer and Claude Code's highlight.js terminal renderer, are accommodated instead by a **display-only** convention: Loment code is fenced as rust, because the two surfaces overlap enough that the colouring comes out right (docs/157). The repository carries one workspace-level instruction file that several agents read (AGENTS.md, read by ZCode, Claude Code and DSH), and the language's single-source skill is **pointed at** from each other agent's own user-level file, Codex's ~/codex/AGENTS.md for instance, together with an environment variable, so that no second copy of the skill can drift (AGENTS.md).

This pattern is the same one the L2 epoch shows (Section 4.4): an interface is narrowed to what the other side can actually consume, and the narrowing is recorded rather than hidden.

B.5 Verification is built for a machine to run, because a machine runs it

An agent developing to completion needs gates that do not depend on a human's judgement, and the project's gates are of that kind.

- **Two independent implementations, byte-identical.** The reference implementation and the self-hosted one must emit byte-identical intermediate code for the same input. The project describes this as the premise that lets the two implementations check each other and as its primary means of regression detection (docs/158). The discipline has teeth: the project's own divergence table records real disagreements found this way, among them a self-hosted pointer cast truncated to 32 bits, and a mixed-width integer comparison that produced illegal code while **both** implementations were consistent with each other. That last entry is why the project writes that "byte-identical" should not have an unrecorded exception" (docs/158).
- **A reproducible release manifest.** A checker recomputes the sha256 of 98 artifacts and compares them, so that a third party can verify a checkout in one command (docs/152).
- **A commit gate.** A scanner sits on commits and hard-blocks them, leaving work in the working tree (docs/140 §10, docs/152). It is worth recording that this scanner "did not reach a complete conclusion", and that the project therefore **does not claim to be secure**: the limitation is written into the audit tooling itself (docs/160, docs/203, tools/loment_audit.py).

B.6 Models are also an object of measurement

Models appear in this project not only as tools but as a quantity to be measured. One milestone arm, called "the LLM arm", was actually run, using two small local models served by Ollama (qwen3:4b and llama3.2:3b) against a corpus whose digest is shared with the toolchain arm, so that the two can be compared (docs/145).

B.7 The through-line

Two of this design's most constrained decisions look like one decision made twice. The block epoch is narrowed to a single store because the in-package linker has no select (Section 4.4), and the language surface is narrowed to a Rust subset because the agent that writes it has no other corpus. Both are cases where the design pays a price to stay within its tools and writes the price down. What this appendix adds is only the identity of one of those tools.

Bibliography

- [1] M. Tofte and J.-P. Talpin, “Region-Based Memory Management,” *Information and Computation*, vol. 132, no. 2, pp. 109–176, 1997.
- [2] A. Reinking, N. Xie, L. de Moura, and D. Leijen, “Perceus: Garbage Free Reference Counting with Reuse,” in *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '21)*, 2021, pp. 96–111.
- [3] H. G. Baker, “NREVERSAL of Fortune — The Thermodynamics of Garbage Collection,” in *Memory Management (IWMM '92)*, in Lecture Notes in Computer Science, vol. 637. Springer, 1992, pp. 507–524.
- [4] D. F. Bacon, P. Cheng, and V. T. Rajan, “A Unified Theory of Garbage Collection,” in *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '03)*, 2003, pp. 50–68.
- [5] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman, Hall/CRC, 2011.
- [6] D. Ungar, “Generation Scavenging: A Non-Disruptive High Performance Storage Reclamation Algorithm,” in *Proceedings of the First ACM SIGSOFT/SIGPLAN Software Engineering Symposium on Practical Software Development Environments*, 1984, pp. 157–167.
- [7] E. W. Dijkstra, L. Lamport, A. J. Martin, C. S. Scholten, and E. F. M. Steffens, “On-the-Fly Garbage Collection: An Exercise in Cooperation,” *Communications of the ACM*, vol. 21, no. 11, pp. 966–975, 1978.
- [8] Y. Wu, “Garbage Entropy and the Limits of Bounded-Memory Computation.” 2026. doi: 10.5281/zenodo.20252480.
- [9] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” *IBM Journal of Research and Development*, vol. 5, no. 3, pp. 183–191, 1961.
- [10] C. H. Bennett, “Logical Reversibility of Computation,” *IBM Journal of Research and Development*, vol. 17, no. 6, pp. 525–532, 1973.
- [11] J. M. R. Parrondo, J. M. Horowitz, and T. Sagawa, “Thermodynamics of Information,” *Nature Physics*, vol. 11, pp. 131–139, 2015.